

Curso de Especialización Ciberseguridad en las TIC

Práctica 2. Despliegue de un NIDS/NIPS en la red interna/corporativa. Integración en Wazuh.

Objetivos

- Instalar y configurar Suricata en Ubuntu Server.
- Personalizar las reglas para detectar/bloquear posible tráfico de red malicioso.
- Configurar el SIEM/XDR Wazuh.
- Integrar Suricata con Wazuh.

Resumen

Autora: Marina del Pilar López Oliva

Marzo/Abril 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	25/03/2026	Adaptación de la plantilla.
2.0	12/04/2026	Documentación de la práctica.

Índice de Contenidos

1. Introducción.....	4
2. RA7.d - Bloque 5. 1 Evidencias de la configuración de Wazuh en el contexto de la práctica.....	4
3. RA7.d - Bloque 5. 2 Integración de Agentes.....	7
4. RA7.d - Bloque 5. 3 Detección de la vulnerabilidad Apache Text4Shell.....	9
5. RA7.d - Bloque 5. 4 Alertar de consultas DNS maliciosas.....	10
6. RA7.d - Bloque 5. 5 Pruebas de concepto.....	11
7. RA7.d - Bloque 5. 8 Realizar visualizaciones.....	12
8. RA5.e - Bloque 4. Instalación y configuración de Suricata en Ubuntu Server...	16
9. RA5.e - Bloque 5. Apartado 7. Configurar y monitorizar métricas del servidor Wazh.....	20
10. RA5.d - Bloque 4.....	21
11. RA5.d - Bloque 5. Apartado 6. (Bloqueo de ataque DoS.).....	23
12. Conclusiones.....	25
13. Webgrafía.....	25

1. Introducción.

En esta práctica hemos desplegado un sistema NIDS/NIPS (Network Intrusion Detection/Prevention System) basado en Suricata, integrado completamente con el SIEM/XDR Wazuh. El objetivo principal ha sido implementar contramedidas frente a comportamientos no deseados en la red, específicamente ataques de denegación de servicio (DoS), fuerza bruta FTP, escaneo de puertos y otras amenazas.

El escenario de trabajo ha consistido en una red interna con direccionamiento 192.168.10.0/26, donde se han desplegado cuatro máquinas virtuales en VirtualBox: el servidor Wazuh (SIEM) con IP 192.168.10.10, un servidor web LAMP con WordPress (equipo objetivo) con IP 192.168.10.20, un servidor con Suricata (NIDS/NIPS) con IP 192.168.10.30 y una máquina Kali Linux (equipo atacante) con IP 192.168.10.50. Todas las máquinas se han configurado con IP estática dentro del mismo rango para garantizar la conectividad.

2. RA7.d - Bloque 5. 1 Evidencias de la configuración de Wazuh en el contexto de la práctica.

En este apartado presentamos la configuración general de Wazuh, incluyendo los archivos `ossec.conf` y `local_rules.xml`, que son fundamentales para el correcto funcionamiento del SIEM

Hemos utilizado el servidor Wazuh de la práctica pasada. Se ha configurado la interfaz de red con la IP estática 192.168.10.10/26. El archivo `/etc/netplan/50-cloud-init.yaml` se ha modificado para deshabilitar DHCP y establecer la IP fija.

```
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: no
      addresses: [192.168.10.10/26]
      nameservers:
        addresses:
          - 8.8.8.8
          - 1.1.1.1
      routes:
        - to: default
          via: 192.168.10.1
```

Figura 1. Configuración de red del servidor Wazuh.

Una vez completada la instalación, hemos accedido al panel web mediante <https://192.168.10.10>. El navegador nos muestra una advertencia de certificado autofirmado, que tenemos que aceptar para continuar. Tras introducir las credenciales de administrador, podemos visualizar el dashboard principal de Wazuh, confirmando que el servidor estaba operativo.

El archivo [/var/ossec/etc/ossec.conf](#) contiene la configuración principal del servidor Wazuh. Para esta práctica, hemos añadido la sección de Active Response necesaria para bloquear ataques DoS, tal como se documenta en el capítulo 3.1.4. En la siguiente figura, se puede observar la configuración del comando firewall-drop y la respuesta activa asociada a la regla 100090.

```
<command>
  <name>firewall-drop</name>
  <executable>firewall-drop</executable>
  <expect>srcip</expect>
  <timeout_allowed>yes</timeout_allowed>
</command>
```

Figura 2. Configuración de Active Response en *ossec.conf*.

El comando firewall-drop utiliza el script del mismo nombre ubicado en */var/ossec/active-response/bin/*, que implementa el bloqueo mediante iptables. El parámetro *expect="srcip"* indica que el script espera recibir la dirección IP del atacante, y *timeout_allowed="yes"* permite establecer un tiempo de desbloqueo automático.

Además, hemos configurado el wodle syscollector para la recolección de información del sistema. Este wodle permite monitorizar el hardware, sistema operativo, red, paquetes instalados, puertos y procesos del servidor.

```
<!-- System inventory -->
<wodle name="syscollector">
  <disabled>no</disabled>
  <interval>1h</interval>
  <scan_on_start>yes</scan_on_start>
  <hardware>yes</hardware>
  <os>yes</os>
  <network>yes</network>
  <packages>yes</packages>
  <ports all="no">yes</ports>
  <processes>yes</processes>

  <!-- Database synchronization settings -->
  <synchronization>
    <max_eps>10</max_eps>
  </synchronization>
</wodle>
```

Figura 3. Configuración del wodle syscollector.

El archivo [/var/ossec/etc/rules/local_rules.xml](#) contiene las reglas personalizadas de Wazuh. En este archivo se definieron múltiples reglas para la detección de ataques DoS, vulnerabilidades y métricas de rendimiento.

La regla para la detección de ataques DoS se muestra en la Figura 4. Esta regla, con ID 100001 y nivel 10, se activa cuando Suricata detecta un posible ataque DoS y genera una alerta que es visualizada en el panel de Wazuh.

```
<group name="dos_attack">
  <rule id="100001" level="10">
    <if_sid>2000</if_sid>
    <match>Posible ataque DoS</match>
    <description>ALERTA: Posible ataque DoS detectado por Suricata</description>
    <group>dos,attack,</group>
  </rule>
</group>
```

Figura 4. Regla de detección DoS en local_rules.xml.

La regla utiliza if_sid 2000, que corresponde a eventos generados por Suricata, y busca la cadena "Posible ataque DoS" en el mensaje de la alerta. Esta integración permite que Wazuh actúe como SIEM centralizado, recibiendo y procesando las alertas generadas por el NIDS.

3. RA7.d - Bloque 5. 2 Integración de Agentes.

En este apartado, hemos integrado los agentes necesarios para tener monitorizados todos los equipos de la red. Se han integrado el servidor LAMP (equipo objetivo) y el servidor Suricata (NIDS/NIPS).

En el servidor LAMP hemos instalado el agente Wazuh versión 4.12.0-1. Durante la configuración, hemos añadido la monitorización de logs de MySQL añadiendo un bloque `<localfile>` con `log_format="mysql_log"`, como se aprecia en la Figura 5. El archivo [ossec.conf](#) se ha configurado con la dirección del manager 192.168.10.10 y el puerto 1514. La Figura 6 muestra esta configuración.

```
GNU nano 4.8 /var/ossec/etc/ossec.conf
<localfile>
  <log_format>mysql_log</log_format>
  <location>/var/log/mysql/error.log</location>
</localfile>
```

Figura 5. Monitorización de logs de MySQL.

```
GNU nano 4.8 /var/ossec/etc/ossec.conf
<!--
Wazuh - Agent - Default configuration for ubuntu 20.04
More info at: https://documentation.wazuh.com
Mailing list: https://groups.google.com/forum/#!forum/wazuh
-->

<ossec_config>
  <client>
    <server>
      <address>192.168.10.10</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <config-profile>ubuntu, ubuntu20, ubuntu20.04</config-profile>
    <notify_time>10</notify_time>
    <time-reconnect>60</time-reconnect>
```

Figura 6. Configuración del agente en LAMP.

El registro del agente lo hemos realizado mediante el comando `agent-auth`, como se documenta en el capítulo 2.2.1.5. En la siguiente imagen se muestra el proceso de descarga e instalación del paquete `.deb`. Tras habilitar el servicio con `systemctl enable`, se ha iniciado el agente y se verificó su estado, que aparecía como `active (running)`.

Práctica 2. Despliegue de un NIDS/NIPS en la red interna/corporativa. Integración en Wazuh.

```
administrador@lamp-wpress:~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.12.0-1_amd64.deb && sudo WAZUH-MANAGER='192.168.10.10' WAZUH-AGENT_NAME='LAMP-WordPress' dpkg -i ./wazuh-agent_4.12.0-1_amd64.deb
--2026-04-09 18:17:47-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.12.0-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 10.172.213.111, 10.172.213.29, 10.172.213.42, ...
Connecting to packages.wazuh.com (packages.wazuh.com)|10.172.213.111|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 11963008 (11M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.12.0-1_amd64.deb.1'

wazuh-agent_4.12.0-1_amd 100%[=====>] 11,41M 39,1MB/s in 0,8s

2026-04-09 18:17:47 (39,1 MB/s) - 'wazuh-agent_4.12.0-1_amd64.deb.1' saved [11963008/11963008]

(Leyendo la base de datos ... 112447 ficheros o directorios instalados actualmente.)
Preparando para desempaquetar .../wazuh-agent_4.12.0-1_amd64.deb ...
Desempaquetando wazuh-agent (4.12.0-1) sobre (4.12.0-1) ...
Configurando wazuh-agent (4.12.0-1) ...
Procesando disparadores para systemd (245.4-4ubuntu3.24) ...
administrador@lamp-wpress:~$
```

Figura 7. Instalación del agente Wazuh en LAMP.

```
administrador@lamp-wpress:~$ sudo systemctl daemon-reload
administrador@lamp-wpress:~$ sudo systemctl enable wazuh-agent
administrador@lamp-wpress:~$ sudo systemctl start wazuh-agent
administrador@lamp-wpress:~$ sudo systemctl status wazuh-agent
● wazuh-agent.service - Wazuh agent
   Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-04-09 18:19:14 UTC; 6s ago
     Process: 62528 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 22 (limit: 4592)
   Memory: 223.7M
   CGroup: /system.slice/wazuh-agent.service
           └─62586 /bin/sh active-response/bin/restart.sh agent
             └─62590 /bin/sh /var/ossec/bin/wazuh-control restart
               └─63082 /var/ossec/bin/wazuh-execd
                 └─63092 /var/ossec/bin/wazuh-agentd
                   └─63107 /var/ossec/bin/wazuh-syscheckd
                     └─63119 sleep 1
                       └─63120 /var/ossec/bin/wazuh-logcollector

abr 09 18:19:07 lamp-wpress systemd[1]: Starting Wazuh agent...
abr 09 18:19:07 lamp-wpress env[62528]: Starting Wazuh v4.12.0...
abr 09 18:19:08 lamp-wpress env[62528]: Started wazuh-execd...
abr 09 18:19:09 lamp-wpress env[62528]: Started wazuh-agentd...
abr 09 18:19:10 lamp-wpress env[62528]: Started wazuh-syscheckd...
abr 09 18:19:11 lamp-wpress env[62528]: Started wazuh-logcollector...
abr 09 18:19:12 lamp-wpress env[62528]: Started wazuh-modulesd...
abr 09 18:19:14 lamp-wpress env[62528]: Completed.
abr 09 18:19:14 lamp-wpress systemd[1]: Started Wazuh agent.
lines 1-24/24 (END)
```

Figura 8. Estado activo del agente LAMP.

En el servidor Suricata se descargó e instaló el mismo paquete `wazuh-agent_4.12.0-1_amd64.deb`. Hemos editado el archivo </var/ossec/etc/ossec.conf> para establecer la dirección del manager (192.168.10.10) y añadir dos bloques `<localfile>` para monitorizar los archivos `eve.json` y `fast.log` de Suricata. Esto permite que todas las alertas generadas por Suricata sean enviadas al SIEM.

Práctica 2. Despliegue de un NIDS/NIPS en la red interna/corporativa. Integración en Wazuh.

```
admin@Suricata:~$ wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.12.0-1_amd64.deb && sudo WAZUH_MANAGER='192.168.10.10' WAZUH_AGENT_NAME='suricata' dpkg -i ./wazuh-agent_4.12.0-1_amd64.deb
--2026-04-11 18:13:47-- https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/wazuh-agent_4.12.0-1_amd64.deb
Resolving packages.wazuh.com (packages.wazuh.com)... 18.172.213.29, 18.172.213.42, 18.172.213.111, ...
Connecting to packages.wazuh.com (packages.wazuh.com)|18.172.213.29|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 11963008 (11M) [application/vnd.debian.binary-package]
Saving to: 'wazuh-agent_4.12.0-1_amd64.deb'

wazuh-agent_4.12.0-1_amd64.deb        6%[====>] 814,84K  176KB/s  eta 2m 49s
```

Figura 9. Instalación del agente en Suricata.

```
GNU nano 7.2 /var/ossec/etc/ossec.conf *
<!--
Wazuh - Agent - Default configuration for ubuntu 24.04
More info at: https://documentation.wazuh.com
Mailing list: https://groups.google.com/forum/#!forum/wazuh
-->

<ossec_config>
  <client>
    <server>
      <address>192.168.10.10</address>
      <port>1514</port>
      <protocol>tcp</protocol>
    </server>
    <config-profile>ubuntu, ubuntu24, ubuntu24.04</config-profile>
    <notify_time>10</notify_time>
    <time-reconnect>60</time-reconnect>
    <auto_restart>yes</auto_restart>
    <crypto_method>aes</crypto_method>
  </client>
```

Figura 10. Configuración del manager en Suricata.

```
<localfile>
  <log_format>json</log_format>
  <location>/var/log/suricata/eve.json</location>
</localfile>

<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/suricata/fast.log</location>
</localfile>
```

Figura 11. Monitorización de logs de Suricata.

4. RA7.d - Bloque 5. 3 Detección de la vulnerabilidad Apache Text4Shell.

Para la detección de la vulnerabilidad Apache Text4Shell (CVE-2022-42889), hemos creado una regla personalizada en Wazuh. Esta vulnerabilidad afecta a versiones de Apache Commons Text y permite la ejecución remota de código, siendo crítico detectar cualquier intento de explotación.

La regla implementada en `/var/ossec/etc/rules/local_rules.xml` fue la siguiente:

```
<group name="text4shell">
  <rule id="100200" level="12">
    <match>Text4Shell|CVE-2022-42889</match>
    <description>Vulnerabilidad Apache Text4Shell detectada</description>
    <group>vulnerability,apache,</group>
  </rule>
</group>
```

Figura 12. Regla personalizada para la vulnerabilidad Apache Text4Shell

Esta regla se activa cuando en los logs se detectan patrones relacionados con Text4Shell, generando una alerta de nivel 12 (alta importancia). La monitorización de logs de Apache, configurada previamente en el agente LAMP, permite que estas detecciones lleguen al SIEM. El nivel 12 se justifica por la criticidad de la vulnerabilidad y su potencial para comprometer el servidor web.

5. RA7.d - Bloque 5. 4 Alertar de consultas DNS maliciosas.

Para detectar consultas DNS maliciosas, hemos creado una regla personalizada en Wazuh que monitoriza posibles comunicaciones con servidores de comando y control (C2) o dominios asociados a malware.

La regla implementada fue la siguiente:

```
<group name="malicious_dns">
  <rule id="100300" level="10">
    <if_sid>31000</if_sid>
    <match>malicious|malware|c2|command and control</match>
    <description>Posible consulta DNS maliciosa detectada</description>
    <group>dns,malware,</group>
  </rule>
</group>
```

Figura 13. Regla personalizada para alertar de consultas DNS maliciosas.

Esta regla se basa en el if_sid 31000, que corresponde a eventos DNS capturados por Suricata. Cuando Suricata detecta una consulta DNS sospechosa, genera un evento que es enviado a Wazuh. La regla busca palabras clave como "malicious", "malware", "c2" o "command and control" en el contenido del evento, activando una alerta de nivel 10. Este enfoque permite identificar posibles infecciones por malware que utilizan DNS para comunicarse con sus servidores de comando y control.

6. RA7.d - Bloque 5. 5 Pruebas de concepto.

Siguiendo el capítulo 3.1.3 de los contenidos de la unidad, hemos implementado dos pruebas de concepto: detección de fuerza bruta SSH y detección de comandos maliciosos.

Se ha implementado una regla en Wazuh que activa una alerta cuando se detectan múltiples intentos fallidos de autenticación SSH desde la misma IP origen. La regla creada en local_rules.xml fue la siguiente:

```
<group name="ssh_bruteforce">
  <rule id="100400" level="10" frequency="5" timeframe="60">
    <if_matched_sid>5712</if_matched_sid>
    <same_source_ip />
    <description>Posible ataque de fuerza bruta SSH detectado</description>
    <group>ssh, bruteforce, attack,</group>
  </rule>
</group>
```

Figura 14. Regla de detección de fuerza bruta SSH.

La regla se activa cuando la regla base 5712 (que detecta fallos de autenticación SSH) coincide 3 veces en un período de 30 segundos desde la misma IP origen, generando una alerta de nivel 10.

Para probar la regla, hemos utilizado la herramienta hydra desde Kali. El comando ejecutado fue `hydra -l root -P rockyou.txt ssh://192.168.10.20`, que intenta autenticarse con el usuario root utilizando un diccionario de contraseñas.

```
(kali@kali) ~$ hydra -l root -P /home/kali/Downloads/rockyou.txt ssh://192.168.10.20
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-12 05:19:20
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip waiting)) from a previous session found, ./hydra.restore
re
```

Figura 15. Ataque de fuerza bruta con hydra.

La alerta generada por Wazuh la podemos visualizar en Discover). En el evento se puede observar la IP del atacante (192.168.10.50), el usuario objetivo (root) y la descripción de la regla activada.



Figura 16. Alerta de fuerza bruta SSH en Wazuh.

La alerta generada por Wazuh la podemos visualizar en Discover). En el evento se puede observar la IP del atacante (192.168.10.50), el usuario objetivo (root) y la descripción de la regla activada.

Hemos implementado una segunda regla para detectar la ejecución de comandos maliciosos como nc -l (netcat en modo escucha), ncat, bash -i (shell interactiva) y python -c (ejecución de código Python). Estos comandos son comúnmente utilizados en ataques para establecer reverse shells o persistencia.

La regla implementada fue la siguiente:

```
<group name="malicious_commands">
  <rule id="100500" level="10">
    <match>nc -l|ncat|bash -i|python -c|perl -e</match>
    <description>Comando malicioso detectado en el sistema</description>
    <group>malware,execution,attack,</group>
  </rule>
</group>
```

Figura 17. Regla de detección de comandos maliciosos.

La regla utiliza el operador | para realizar una coincidencia con cualquiera de los patrones especificados, generando una alerta de nivel 10 cuando se detecta alguno de estos comandos en los logs del sistema. La monitorización de procesos se realiza mediante el wodle syscollector configurado en Wazuh.

7. RA7.d - Bloque 5. 8 Realizar visualizaciones.

Hemos creado múltiples dashboards en Wazuh para visualizar los datos recopilados, siguiendo el capítulo 3.1.1 de los contenidos de la unidad. La Figura 18 muestra la pantalla principal de creación de dashboards

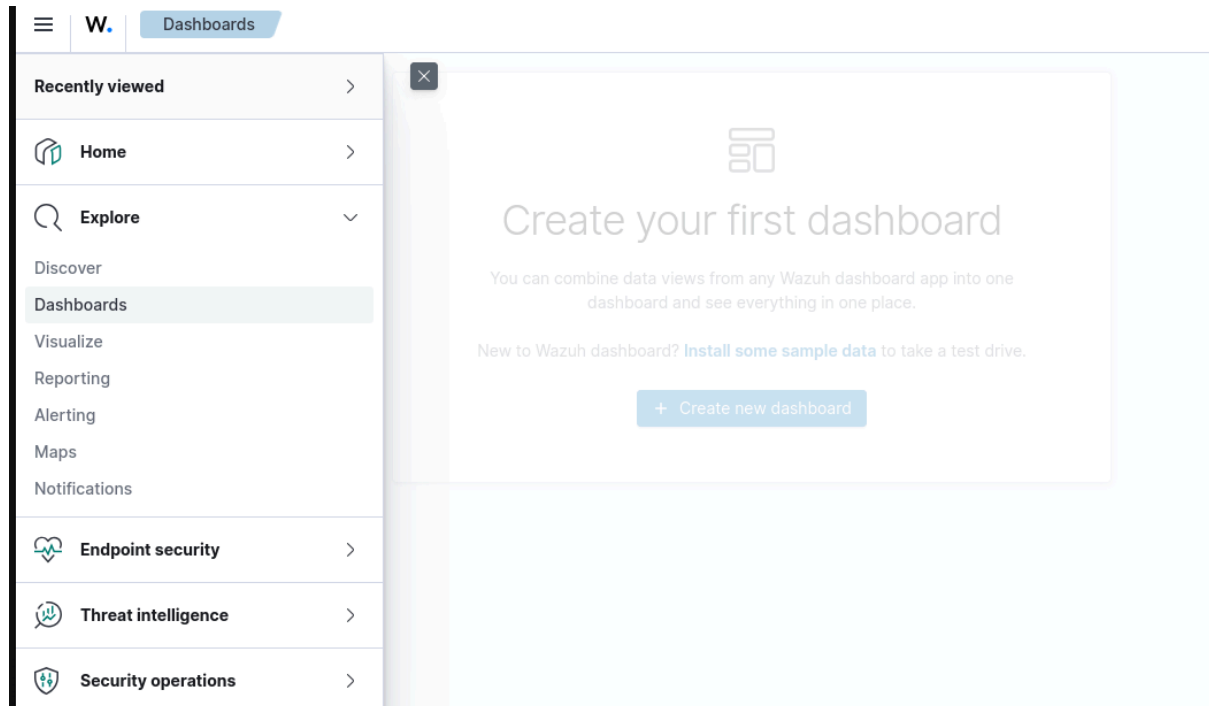


Figura 18. Pantalla de creación de dashboards.

Hemos creado un dashboard con un gráfico de líneas para visualizar la evolución de los ataques DoS detectados. Hemos utilizado el filtro `rule.id:100400 OR rule.groups:dos` para mostrar solo las alertas relacionadas. La Figura 19 muestra la configuración de la visualización con el eje X como timestamp (histograma) y el eje Y como Count.

The image shows a configuration panel for metrics visualization. It is divided into two main sections: 'Metrics' and 'Buckets'. In the 'Metrics' section, there is a dropdown menu currently showing 'Y-axis Count' and a blue '+ Add' button below it. The 'Buckets' section contains a 'X-axis' dropdown menu with a chevron icon on the left and an eye icon with a red 'X' on the right. Below this, there is an 'Aggregation' section with a dropdown menu set to 'Date Histogram' and a 'Date Histogram help' link with an external icon. Underneath is a 'Field' section with a dropdown menu set to 'timestamp'. At the bottom of the panel, there are two buttons: a blue 'X Discard' button on the left and a blue 'Update' button with a refresh icon on the right.

Figura 19. Configuración de visualización de métricas.

Hemos creado un dashboard para monitorizar los logs de acceso y error de Apache, utilizando el filtro `agent.name:LAMP AND data.program:apache2`. La Figura 20 muestra la configuración del filtro aplicado.

The image shows the top part of the Wazuh dashboard interface. It includes a header with a menu icon, the 'W.' logo, and tabs for 'Visualize' and 'Create'. To the right of the tabs are links for 'Inspect', 'Share', and 'Reporting', along with a 'Save' button, a user profile icon with the letter 'a', and a help icon. Below the header, there is a filter bar containing the text 'agent.name:LAMP AND data.program:apache2', a 'DQL' button, a calendar icon with a dropdown arrow, and the text 'Last 24 hours'. To the right of the filter bar are 'Show dates' and 'Refresh' buttons.

Figura 20. Filtro para dashboard de Apache.

Hemos creado un dashboard con visualizaciones de tipo "gauge" para mostrar el promedio de uso de CPU, memoria y disco. La Figura 22 muestra la configuración de métricas con Average bucket y campo timestamp, que resultó en un valor promedio de 4.636.

Práctica 2. Despliegue de un NIDS/NIPS en la red interna/corporativa. Integración en Wazuh.

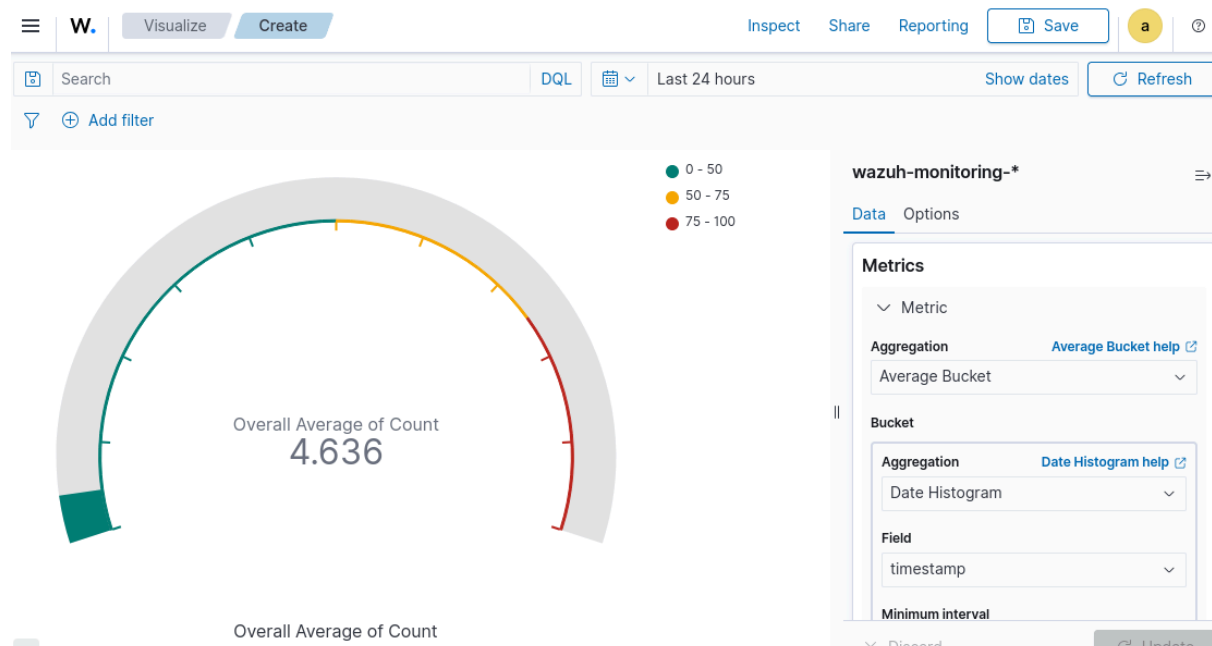
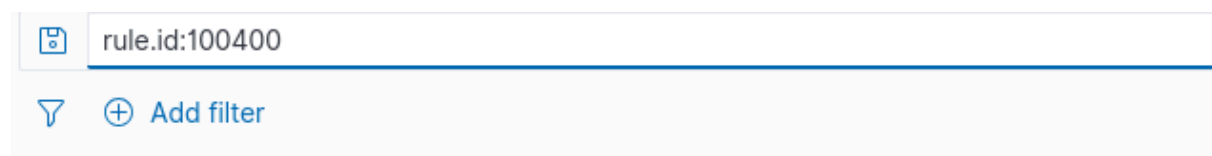


Figura 21. Dashboard de métricas de rendimiento.

Hemos creado un dashboard del tipo “metric” específico para las pruebas de concepto, filtrando por rule.id:100400 (fuerza bruta SSH) y rule.id:100500 (comandos maliciosos).

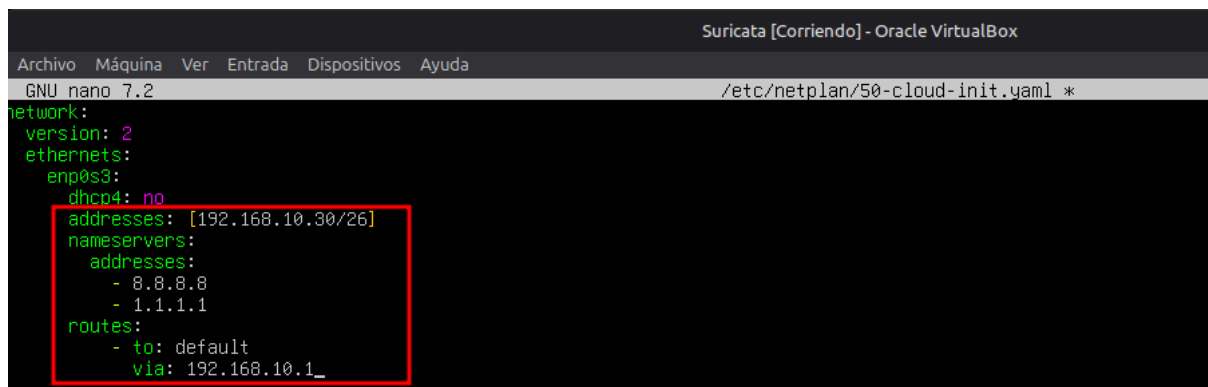


157
Count

Figura 22. Dashboard “metric” para pruebas de conceptos (fuerza bruta SSH).

8. RA5.e - Bloque 4. Instalación y configuración de Suricata en Ubuntu Server.

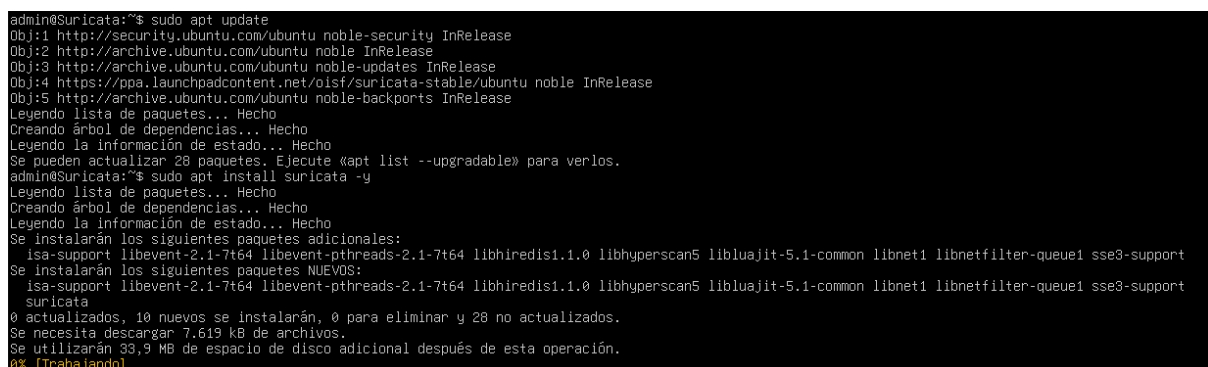
La instalación de Suricata la hemos realizado siguiendo el capítulo 2.2.2 de los contenidos de la unidad. Hemos creado una nueva máquina virtual con Ubuntu Server y se ha configurado con la IP 192.168.10.30/26

A screenshot of a terminal window titled 'Suricata [Corriendo] - Oracle VirtualBox'. The terminal shows the GNU nano 7.2 editor editing the file /etc/netplan/50-cloud-init.yaml. The configuration is for a network interface named enp0s3. A red rectangle highlights the following configuration: addresses: [192.168.10.30/26], nameservers: addresses: - 8.8.8.8, - 1.1.1.1, routes: - to: default, via: 192.168.10.1_.

```
Suricata [Corriendo] - Oracle VirtualBox
Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml *
network:
  version: 2
  ethernet:
    enp0s3:
      dhcp4: no
      addresses: [192.168.10.30/26]
      nameservers:
        addresses:
          - 8.8.8.8
          - 1.1.1.1
      routes:
        - to: default
          via: 192.168.10.1_
```

Figura 23. Configuración de red de Suricata.

Hemos añadido el repositorio PPA de Suricata estable mediante add-apt-repository ppa:oisf/suricata-stable. Posteriormente, hemos actualizado la lista de paquetes e instalado Suricata junto con sus dependencias, incluyendo libnetfilter-queue1, necesaria para el modo IPS.

A screenshot of a terminal window showing the installation of Suricata. The user runs 'sudo apt update' and 'sudo apt install suricata -y'. The output shows the update of package lists and the installation of Suricata along with its dependencies. A red rectangle highlights the list of packages to be installed, including libnetfilter-queue1.

```
admin@Suricata:~$ sudo apt update
Obj:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:2 http://archive.ubuntu.com/ubuntu noble InRelease
Obj:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:4 https://ppa.launchpadcontent.net/oisf/suricata-stable/ubuntu noble InRelease
Obj:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 28 paquetes. Ejecute «apt list --upgradable» para verlos.
admin@Suricata:~$ sudo apt install suricata -y
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  isa-support libevent-2.1-7t64 libevent-pthreads-2.1-7t64 libhiredis1.1.0 libhyperscan5 liblua5.1-common libnet1 libnetfilter-queue1 sse3-support
Se instalarán los siguientes paquetes NUEVOS:
  isa-support libevent-2.1-7t64 libevent-pthreads-2.1-7t64 libhiredis1.1.0 libhyperscan5 liblua5.1-common libnet1 libnetfilter-queue1 sse3-support
  suricata
0 actualizados, 10 nuevos se instalarán, 0 para eliminar y 28 no actualizados.
Se necesita descargar 7.619 kB de archivos.
Se utilizarán 33,9 MB de espacio de disco adicional después de esta operación.
0% [Trabajando]
```

Figura 24. Instalación del paquete Suricata.

Hemos editado el archivo [/etc/suricata/suricata.yaml](#) para definir la red interna (HOME_NET: "[192.168.10.0/26]"), configurar la interfaz de red enp0s3 en la sección af-packet y habilitar la recarga de reglas en vivo con rule-reload: true. La Figura 24 muestra la configuración de HOME_NET y la Figura 25 la configuración de la interfaz af-packet.


```
GNU nano 7.2 /etc/suricata/suricata.yaml *
vars:
# more specific is better for alert accuracy and performance
address-groups:
HOME_NET: "[192.168.10.0/26]"
#HOME_NET: "[192.168.0.0/16]"
#HOME_NET: "[10.0.0.0/8]"
#HOME_NET: "[172.16.0.0/12]"
#HOME_NET: "any"

EXTERNAL_NET: "!$HOME_NET"
#EXTERNAL_NET: "any"

HTTP_SERVERS: "$HOME_NET"
```

Figura 25. Configuración de HOME_NET en Suricata.

```
GNU nano 7.2 /etc/suricata/suricata.yaml *
##
# Linux high speed capture support
af-packet:
- interface: enp0s3
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow.
  cluster-id: 99
  # Default AF_PACKET cluster type. AF_PACKET can load balance per flow or per hash.
  # This is only supported for Linux kernel > 3.1
  # possible value are:
  # * cluster_flow: all packets of a given flow are sent to the same socket
  # * cluster_cpu: all packets treated in kernel by a CPU are sent to the same socket
  # * cluster_qm: all packets linked by network card to a RSS queue are sent to the same
  # socket. Requires at least Linux 3.14.
  # * cluster_ebpf: eBPF file load balancing. See doc/userguide/capture-hardware/ebpf-xdp.rst for
  # more info.
```

Figura 26. Configuración de la interfaz af-packet.

Hemos ejecutado el comando `suricata-update` para descargar el conjunto de reglas de Emerging Threats. Este paso es fundamental para que Suricata pueda detectar amenazas conocidas, ya que el conjunto de reglas se actualiza constantemente con nuevas firmas.

Suricata se ha configurado en modo IPS añadiendo el parámetro `-q 0` al comando de inicio. Hemos verificado que el sistema soportara NFQUEUE mediante `suricata --build-info | grep NFQ`, que mostró `NFQueue support: yes`, como se aprecia en la Figura 26. Posteriormente, se ha modificado el archivo de servicio de Suricata para incluir `-q 0`, como se muestra en la Figura 27.

```
admin@Suricata:~$ sudo suricata --build-info | grep NFQ
Features: NFQ PCAP_SET_BUFF AF_PACKET HAVE_PACKET_FANOUT LIBCAP_NG LIBNET1.1 HAVE_HTTP_URI_NORMALIZE_HOOK PCRE_JIT HAVE_NSS HTTP2_DECOMPRESSION HAVE_LUA HAVE_JA3
HAVE_JA4 HAVE_LIGANSSON TLS TLS_C11 MAGIC RUST POPCNT64
NFQueue support: yes
admin@Suricata:~$
```

Figura 27. Verificación de soporte NFQUEUE.

Práctica 2. Despliegue de un NIDS/NIPS en la red interna/corporativa. Integración en Wazuh.

```
GNU nano 7.2 /usr/lib/systemd/system/suricata.service *
[Unit]
Description=Suricata IDS/IPS/NSM/FW daemon
After=network.target network-online.target
Requires=network-online.target
Documentation=man:suricata(8) man:suricatasc(8)
Documentation=https://suricata.io/documentation/

[Service]
#User=suricata
#Group=suricata
#Environment=LD_PRELOAD=/usr/lib/libc_mallot_minimal.so.4
PIDFile=/run/suricata.pid
ExecStartPre=/bin/rm -f /run/suricata.pid
ExecStart=/usr/bin/suricata -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid -q 0
ExecReload=/bin/kill -USR2 $MAINPID
ExecStop=/usr/bin/suricatasc -c shutdown
Restart=on-failure_
ProtectSystem=full
ProtectHome=true

[Install]
WantedBy=multi-user.target
```

Figura 28. Configuración de Suricata en modo IPS.

Hemos añadido reglas en `/etc/ufw/before.rules` para redirigir el tráfico a la cola NFQUEUE, tal como se documenta en el capítulo 2.2.2. Tras aplicar los cambios, se reinició Suricata y se verificó que el proceso incluyera `-q 0`.

```
GNU nano 7.2 /etc/ufw/before.rules *
-A ufw-before-forward -p icmp --icmp-type echo-request -j ACCEPT

# allow dhcp client to work
-A ufw-before-input -p udp --sport 67 --dport 68 -j ACCEPT

#
# ufw-not-local
#
-A ufw-before-input -j ufw-not-local

# if LOCAL, RETURN
-A ufw-not-local -m addrtype --dst-type LOCAL -j RETURN

# if MULTICAST, RETURN
-A ufw-not-local -m addrtype --dst-type MULTICAST -j RETURN

# if BROADCAST, RETURN
-A ufw-not-local -m addrtype --dst-type BROADCAST -j RETURN

# all other non-local packets are dropped
-A ufw-not-local -m limit --limit 3/min --limit-burst 10 -j ufw-logging-deny
-A ufw-not-local -j DROP

# allow MULTICAST mDNS for service discovery (be sure the MULTICAST line above
# is uncommented)
-A ufw-before-input -p udp -d 224.0.0.251 --dport 5353 -j ACCEPT

# allow MULTICAST UPnP for service discovery (be sure the MULTICAST line above
# is uncommented)
-A ufw-before-input -p udp -d 239.255.255.250 --dport 1900 -j ACCEPT

# Reglas para suricata en modo IPS
-I ufw-before-input 1 -p tcp --dport 22 -j ACCEPT
-I ufw-before-output 1 -p tcp --sport 22 -j ACCEPT

-A ufw-before-input -j NFQUEUE --queue-num 0
-A ufw-before-output -j NFQUEUE --queue-num 0
-A ufw-before-forward -j NFQUEUE --queue-num 0

-A ufw-before-input -i lo -j ACCEPT
-A ufw-before-output -o lo -j ACCEPT

# don't delete the 'COMMIT' line or these rules won't be processed
COMMIT
```

Figura 29. Reglas NFQUEUE en `ufw/before.rules`.

Práctica 2. Despliegue de un NIDS/NIPS en la red interna/corporativa. Integración en Wazuh.

```
admin@Suricata:~$ sudo systemctl daemon-reload
admin@Suricata:~$ sudo systemctl start suricata.service
admin@Suricata:~$ sudo systemctl status suricata.service
• suricata.service - Suricata IDS/IPS/NSM/FW daemon
  Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
  Active: active (running) since Sat 2026-04-11 17:56:10 UTC; 6s ago
    Docs: man:suricata(8)
           man:suricata-sc(8)
           https://suricata.io/documentation/
  Process: 5572 ExecStartPre=/bin/rm -f /run/suricata.pid (code=exited, status=0/SUCCESS)
 Main PID: 5574 (Suricata-Main)
   Tasks: 1 (limit: 2263)
  Memory: 182.8M (peak: 182.8M)
    CPU: 6.612s
  CGroup: /system.slice/suricata.service
          └─5574 /usr/bin/suricata -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid -q 0

abr 11 17:56:10 Suricata systemd[1]: Starting suricata.service - Suricata IDS/IPS/NSM/FW daemon...
abr 11 17:56:10 Suricata systemd[1]: Started suricata.service - Suricata IDS/IPS/NSM/FW daemon.
abr 11 17:56:10 Suricata suricata[5574]: i: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
admin@Suricata:~$ sudo systemctl status suricata.service | grep "Main PID" -A 2
Main PID: 5574 (Suricata-Main)
   Tasks: 10 (limit: 2263)
  Memory: 412.2M (peak: 412.4M)
admin@Suricata:~$
```

Figura 30. Suricata activo en modo IPS.

Hemos creado el archivo [/etc/suricata/rules/iescamp.rules](#) con las reglas personalizadas. Posteriormente, se ha configurado Suricata para cargar este archivo añadiendo la ruta en la sección rule-files del archivo suricata.yaml.

```
GNU nano 7.2 /etc/suricata/rules/iescamp.rules *
# Bloqueo DoS
drop tcp any any -> $HOME_NET 80 (flags: S; msg: "BLOQUEO: Posible ataque DoS SYN Flood"; flow:stateless; detection_filter:track by_dst, count 20, seconds 10;

# Bloqueo de fuerza bruta FTP
drop tcp any any -> $HOME_NET 21 (msg: "BLOQUEO: Posible fuerza bruta FTP"; flow:stateless; detection_filter:track by_src, count 10, seconds 30; sid: 1000002;

# Bloqueo escaneo de puertos
drop tcp any any -> $HOME_NET any (flags: S; msg: "BLOQUEO: Escaneo de puertos detectado"; flow:stateless; detection_filter:track by_src, count 100, seconds 10;

# Detecta acceso a youtube.com
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"DETECCION: Acceso a youtube.com"; http.host; content:"youtube.com";_sid: 1000004; rev:1)
```

Figura 31. Reglas personalizadas en iescamp.rules.

```
GNU nano 7.2 /etc/suricata/suricata.yaml *
##
## Configure Suricata to load Suricata-Update managed rules.
##
default-rule-path: /var/lib/suricata/rules
rule-files:
- suricata.rules
- /etc/suricata/rules/iescamp.rules
##
## Auxiliary configuration files.
##
```

Figura 32. Configuración de rule-files en suricata.yaml.

Hemos ejecutado `suricata -T -c /etc/suricata/suricata.yaml -v` para validar la sintaxis de las reglas. La salida mostró 49538 rules successfully loaded, 0 rules failed, confirmando que todas las reglas eran correctas.

```
admin@Suricata:~$ sudo suricata -T -c /etc/suricata/suricata.yaml -v
Notice: suricata: This is Suricata version 8.0.4 RELEASE running in SYSTEM mode
Info: cpu: CPUs/cores online: 2
Info: suricata: Running suricata under test mode
Info: suricata: Setting engine mode to IDS mode by default
Info: exception-policy: master exception-policy set to: auto
Info: suricata: Preparing unexpected signal handling
Info: logopenfile: fast output device (regular) initialized: fast.log
Info: logopenfile: eve-log output device (regular) initialized: eve.json
Info: logopenfile: stats output device (regular) initialized: stats.log
Info: detect: 2 rule files processed. 49533 rules successfully loaded, 0 rules failed, 0 rules skipped
Info: threshold-config: Threshold config parsed: 0 rule(s) found
Info: detect: 49538 signatures processed. 1275 are IP-only rules, 4481 are inspecting packet payload, 43544 inspect application layer, 110 are decoder event only
Warning: detect: rule 1000001: SYN-only to port(s) 00:80 w/o direction specified, disabling for toclient direction
Notice: suricata: Configuration provided was successfully loaded. Exiting.
admin@Suricata:~$
```

Figura 33. Verificación exitosa de reglas

9. RA5.e - Bloque 5. Apartado 7. Configurar y monitorizar métricas del servidor Wazh

Hemos añadido tres bloques `<localfile>` con `log_format="command"` en `/var/ossec/etc/ossec.conf`. El primer comando monitoriza el uso de CPU mediante `top`, el segundo calcula el porcentaje de memoria RAM utilizada con `free`, y el tercero obtiene el porcentaje de espacio en disco con `df`.

```
<localfile>
  <log_format>command</log_format>
  <command> top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1</command>
  <frequency>300</frequency>
</localfile>

<localfile>
  <log_format>command</log_format>
  <command>free | grep Mem | awk '{print ($3/$2) * 100}'</command>
  <frequency>300</frequency>
</localfile>

<localfile>
  <log_format>command</log_format>
  <command>df -h / | tail -1 | awk '{print $5}' | tr -d '%'</command>
  <frequency>300</frequency>
</localfile>_
```

Figura 34. Monitorización de métricas en `ossec.conf`

Se crearon reglas en `local_rules.xml` para generar alertas cuando las métricas superaran los umbrales establecidos: CPU superior al 75%, RAM superior al 60% y disco superior al 75%. Estas reglas, con nivel 7, permiten al equipo de seguridad identificar posibles problemas de rendimiento antes de que afecten a la operación del SIEM.

```
<group name="metrics">
  <rule id="100100" level="7">
    <match>CPU used</match>
    <description>CPU usage > 75%</description>
    <group>metrics,cpu,</group>
  </rule>
  <rule id="100101" level="7">
    <match>RAM used</match>
    <description>RAM usage > 60%</description>
    <group>metrics,memory,</group>
  </rule>
  <rule id="100102" level="7">
    <match>Disk used</match>
    <description>Disk space > 75%</description>
    <group>metrics,disc,</group>
  </rule>
</group>
```

Figura 35. Reglas para alertas de rendimiento.

10. RA5.d - Bloque 4.

En este apartado se describen las contramedidas implementadas en Suricata como parte del bloque 4 de la práctica, específicamente las reglas de bloqueo (DROP) configuradas para mitigar comportamientos no deseados en la red.

Hemos implementado tres reglas con acción drop (bloqueo) en el archivo `/etc/suricata/rules/iescamp.rules`:

- Regla 1 – Bloqueo de ataques DoS (sid:1000001): `drop tcp any any -> $HOME_NET 80 (flags: S; msg:"BLOQUEO: Posible ataque DoS SYN Flood"; flow:stateless; detection_filter:track by_dst, count 20, seconds 10; sid:1000001; rev:1;)`. Esta regla bloquea inundaciones de paquetes SYN dirigidas al puerto 80 del servidor web. El filtro de detección activa el bloqueo cuando se superan 20 paquetes en 10 segundos desde cualquier origen hacia el mismo destino.
- Regla 2 – Bloqueo de fuerza bruta FTP (sid:1000002): `drop tcp any any -> $HOME_NET 21 (msg:"BLOQUEO: Posible fuerza bruta FTP"; flow:stateless; detection_filter:track by_src, count 10, seconds 30; sid:1000002; rev:1;)`. Esta

Práctica 2. Despliegue de un NIDS/NIPS en la red interna/corporativa. Integración en Wazuh.

regla bloquea intentos de fuerza bruta contra el servicio FTP, activándose tras 10 intentos en 30 segundos desde la misma IP origen. El seguimiento se realiza por IP origen (track by_src) para identificar al atacante.

- Regla 3 – Bloqueo de escaneo de puertos (sid:1000003): *drop tcp any any -> \$HOME_NET any (flags: S; msg:"BLOQUEO: Escaneo de puertos detectado"; flow:stateless; detection_filter:track by_src, count 100, seconds 10; sid:1000003; rev:1;)*. Esta regla bloquea escaneos de puertos mediante paquetes SYN, activándose cuando se superan 100 paquetes en 10 segundos desde la misma IP origen.

También se ha implementado una regla para alertar sobre el acceso a youtube.com.

```
GNU nano 7.2 /etc/suricata/rules/iescamp.rules *
# Bloqueo DoS
drop tcp any any -> $HOME_NET 80 (flags: S; msg: "BLOQUEO: Posible ataque DoS SYN Flood"; flow:stateless; detection_filter:track by_dst, count 20, seconds 10; sid: 1000000; rev: 1;)

# Bloqueo de fuerza bruta FTP
drop tcp any any -> $HOME_NET 21 (msg: "BLOQUEO: Posible fuerza bruta FTP"; flow:stateless; detection_filter:track by_src, count 10, seconds 30; sid: 1000002; rev: 1;)

# Bloqueo escaneo de puertos
drop tcp any any -> $HOME_NET any (flags: S; msg: "BLOQUEO: Escaneo de puertos detectado"; flow:stateless; detection_filter:track by_src, count 100, seconds 10; sid: 1000003; rev: 1;)

# Detecta acceso a youtube.com
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"DETECCION: Acceso a youtube.com"; http.host; content:"youtube.com"; sid: 1000004; rev:1;)
```

Figura 36. Contramedidas implementadas en Suricata.

Durante las pruebas de ataque, Suricata registró los bloqueos en el archivo /var/log/suricata/fast.log. La Figura 36 muestra las entradas generadas durante un ataque de escaneo de puertos y DoS, donde se aprecian las alertas de bloqueo con las reglas 1000003 y 1000001.

```
admin@Suricata:~$ sudo tail -f /var/log/suricata/fast.log
04/11/2026-20:47:58.667605 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.30:4362 -> 192.168.10.10:1514
04/11/2026-20:47:59.693208 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.30:4362 -> 192.168.10.10:1514
04/11/2026-20:48:00.718382 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.30:4362 -> 192.168.10.10:1514
04/11/2026-20:48:01.739217 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.30:4362 -> 192.168.10.10:1514
04/11/2026-20:48:03.789324 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.30:4362 -> 192.168.10.10:1514
04/11/2026-20:48:07.819449 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.30:4362 -> 192.168.10.10:1514
04/11/2026-20:48:16.269510 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.30:4362 -> 192.168.10.10:1514
04/11/2026-20:48:26.727672 [Drop] [**] [1:1000001:1] BLOQUEO: Posible ataque DoS SYN Flood [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.50:3796 -> 192.168.10.20:80
04/11/2026-20:48:26.727672 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.50:3796 -> 192.168.10.20:80
04/11/2026-20:48:32.658299 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.30:4362 -> 192.168.10.10:1514
04/11/2026-20:49:04.997624 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.30:4362 -> 192.168.10.10:1514
04/11/2026-20:49:06.167707 [Drop] [**] [1:1000002:1] BLOQUEO: Posible fuerza bruta FTP [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.50:60470 -> 192.168.10.20:21
04/11/2026-20:49:06.167707 [Drop] [**] [1:1000003:1] BLOQUEO: Escaneo de puertos detectado [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.50:60470 -> 192.168.10.20:21
```

Figura 37. Alertas de bloqueo en fast.log.

En la imagen se puede observar el formato de las entradas: timestamp, acción [Drop], ID de la regla, mensaje descriptivo, protocolo, IP origen y puerto, e IP destino y puerto. Por ejemplo:

```
04/11/2026-20:45:43.684067 [Drop] [**] [1:1000002:1] BLOQUEO: Posible fuerza bruta FTP [**] [Classification: (null)] [Priority: 3] {TCP} 192.168.10.50:41868 -> 192.168.10.20:21
```

Figura 38. Alerta de bloqueo por fuerza bruta durante un ataque.

Desde Kali, hemos lanzado un ataque de inundación SYN contra el servidor web mediante `hping3 -p 80 -S --flood 192.168.10.20`. Se puede observar que no se reciben respuestas (modo flood).

```
(kali@kali)-[~]
$ sudo hping3 -p 80 -S --flood 192.168.10.20
HPING 192.168.10.20 (eth0 192.168.10.20): S set, 40 headers + 0 data bytes
hping in flood mode, no replies will be shown
```

Figura 39. Ataque DoS desde Kali con hping3.

Durante el ataque, cuando se intentó acceder al servidor web con `curl http://192.168.10.20`, obtenemos un error de conexión `Could not connect to server`. Esto confirma que el bloqueo implementado por Suricata fue efectivo, impidiendo que el tráfico malicioso alcanzara al servidor web.

```
$ curl http://192.168.10.20
curl: (28) Failed to connect to 192.168.10.20 port 80 after 134006 ms: Could not connect to server
```

Figura 40. Curl fallando durante el ataque DoS.

11. RA5.d - Bloque 5. Apartado 6. (Bloqueo de ataque DoS.)

En este apartado se describen las implementaciones realizadas en Wazuh XDR como contramedidas para el bloqueo de ataques DoS, complementando la configuración de Suricata.

Hemos configurado el módulo Active Response en Wazuh añadiendo en `/var/ossec/etc/ossec.conf` el comando `firewall-drop` y una respuesta activa para la regla 100400.

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100000</rules_id>
  <timeout>1800</timeout>
</active-response>
```

Figura 41. Configuración de Active Response para bloqueo DoS.

La configuración incluye:

- Comando firewall-drop: Ejecuta el script que añade reglas de iptables para bloquear la IP del atacante.
- <location>local</location>: El bloqueo se aplica en el agente que generó el evento (Suricata).
- <rules_id>100090</rules_id>: La respuesta se activa cuando se dispara la regla 100090 (detección DoS).
- <timeout>1800</timeout>: La IP queda bloqueada durante 1800 segundos (30 minutos), tras los cuales se desbloquea automáticamente.

Cuando Suricata detecta y bloquea un ataque DoS, genera un evento que es enviado a Wazuh. La regla 100400 en Wazuh captura este evento y activa la respuesta activa, que ejecuta el script firewall-drop en el agente Suricata. Este script añade una regla en iptables que bloquea todo el tráfico procedente de la IP atacante, complementando el bloqueo a nivel de aplicación que ya realiza Suricata.

Las alertas generadas por la regla 100400 se visualizaron en el panel de Wazuh. La Figura 40 muestra un evento con la alerta de escaneo de puertos (sid:1000003) y la acción blocked, confirmando que Wazuh registró correctamente el bloqueo realizado por Suricata.

```
> Apr 12, 2026 @ 12:05:39.363 input.type: log agent.ip: 192.168.10.30 agent.name: Suricata agent.id: 003 manager.name: Wazu
h-IesCamp1 data.ip_v: 4 data.src_ip: 192.168.10.50 data.src_port: 17368 data.event_type: alert
data.alert.severity: 3 data.alert.signature_id: 1000003 data.alert.rev: 1 data.alert.gid: 1
data.alert.signature: BLOQUEO: Escaneo de puertos detectado data.alert.action: blocked
data.flow_id: 629675127945182.000000 data.dest_ip: 192.168.10.20 data.proto: TCP
> Apr 12, 2026 @ 12:05:39.362 input.type: log agent.ip: 192.168.10.30 agent.name: Suricata agent.id: 003 manager.name: Wazu
```

Figura 42. Alerta de bloqueo en Wazuh Discover.

12. Conclusiones.

La implementación del NIDS/NIPS con Suricata integrado en Wazuh ha resultado exitosa, permitiendo detectar y bloquear en tiempo real ataques de denegación de servicio, fuerza bruta FTP, escaneo de puertos y otras amenazas. El uso de Wazuh como SIEM centralizado ha facilitado la correlación de eventos y la visualización de alertas a través de dashboards personalizados.

Durante el desarrollo de la práctica he enfrentado diversos desafíos técnicos. El principal obstáculo fue la configuración del modo IPS en Suricata dentro del entorno de VirtualBox, ya que por defecto el tráfico entre máquinas virtuales no pasa a través de Suricata a menos que se configure como puerta de enlace. Este problema se resolvió configurando adecuadamente las reglas de iptables y NFQUEUE, tal como se documenta en el capítulo 2.2.2. Además, tanto en el servidor web (la máquina virtual de LAMP) y en el Kali se puso como puerta de enlace la IP del Suricata, para simular un entorno de red correcto. Ya que en la realidad debería estar entre el mikrotik y el switch correspondiente, así el tráfico tiene que viajar sí o sí por el Suricata.

Otro desafío significativo fue la comunicación entre el agente de Suricata y el servidor Wazuh, que inicialmente mostraba un estado "disconnected". Tras verificar la compatibilidad de versiones (ambos en v4.12.0), los puertos de red (1514 y 1515) y los logs del sistema, se determinó que el problema residía en el registro del agente, que se resolvió mediante una reinstalación limpia y el uso de agent-auth -f. Además, también se tuvo que comprobar el firewall correspondiente al Suricata, añadiendo la comunicación a dichos puertos.

13. Webgrafía.

Capítulo 2.2.1 de los contenidos de la UT.

Capítulo 2.2.2 de los contenidos de la UT.

Capítulo 3.1 de los contenidos de la UT.

Capítulo 3.1.1 de los contenidos de la UT.

Capítulo 3.1.2 de los contenidos de la UT.

Capítulo 3.1.3 de los contenidos de la UT.

Capítulo 3.1.4 de los contenidos de la UT.

Práctica 2. Despliegue de un NIDS/NIPS en la red interna/corporativa. Integración en Wazuh.

Laboratorio de los contenidos de la UT.

[\[Documentación oficial de Wazuh\]](#)

[\[Documentación oficial de Suricata\]](#)